

SOLID 원칙 보고서

예비단원 김민찬

1. Single Responsibility Principle - 클래스는 단 하나의 책임만 가져야 한다

SRP의 내용은 정리하면 다음과 같다.

하나의 클래스는 한가지 역할만 부여받아야 한다. 예를 들면, 공부와 운동 두 가지 역할이 있을 때, person이라는 하나의 클래스에 공부와 운동 두 가지 기능을 구현해서는 안 된다.

이 경우, 공부의 역할을 맡는 클래스 study와 운동의 역할을 맡는 클래스 exercise를 따로 정의하고, 이를 수행하는 역할의 클래스 person을 정의하여 역할을 수행하여야 한다.

이러한 원칙의 이유는 다음과 같다.

먼저, 하나의 클래스에 여러 가지 역할을 부여하게 되면, 클래스의 코드가 복잡해진다. 또한, 동시에 각 코드의 기능을 이해하기 어렵다. 그리고 한 역할의 코드를 수정할 때, 다른 역할이 의도치 않은 문제를 만들 수 있고, 테스트하기에도 어렵다.

반대로, 역할에 따라 클래스를 나누어 작성하게 되면, 필요한 역할만을 가져다 사용할 수 있기 때문에 유연하게 사용이 가능하다.

```
User saved to database: kmc
Welcome email sent to: 1234@gmail.com
Logging activity for user: kmc

C:\Users\USER\Documents\KwangWoon\ROBIT\C++
되었습니다.
이 창을 닫으려면 아무 키나 누르세요 ...
```

srp.cpp 동작 결과

2. Open/Closed Principle - 요소의 확장에는 열려있고, 수정에는 닫혀있어야 한다

OCP의 내용은 정리하면 다음과 같다.

새로운 기능을 추가해야 할 때, 기존에 작성된 코드는 건드리지 않고 새로운 코드를 덧붙이는 방식으로 기능을 확장할 수 있어야 한다. 즉, "확장"은 얼마든지 가능해야 하지만, 그 확장을 위해 이미 완성된 코드를 "수정"하는 일은 없어야 한다.

예를 들면, person 클래스가 수행하는 역할로 공부(study)와 운동(exercise)이 있다고 할 때, 이후 독서(reading)라는 역할이 추가된다면, 기존의 study, exercise 클래스나 이를 사용하는 person 클래스의 코드를 고치는 것이 아니라, reading이라는 새로운 클래스를 하나 추가하는 것만으로 기능이 확장되어야 한다. 이를 위해서는 study, exercise, reading이 공통으로 따르는 상위의 역할(예: activity라는 인터페이스)을 미리 정의해두고, person은 그 상위 역할에만 의존하도록 설계해야 한다.

이러한 원칙의 이유는 다음과 같다.

이미 검증되어 잘 동작하고 있는 기존 코드를 요구사항이 추가될 때마다 계속 수정하게 되면, 그 과정에서 의도치 않은 버그가 발생할 위험이 커진다. 또한 수정된 부분이 다른 기존 기능에 영향을 주지 않는지 매번 다시 확인해야 하므로 유지보수 비용이 커진다. 반대로 기존 코드는 그대로 둔 채 새로운 코드만 추가하는 구조라면, 새 기능을 붙이는 과정에서 기존 기능이 깨질 위험이 훨씬 줄어들고, 변경 이력을 관리하기도 수월해진다.

```
Generating PDF report...
Generating HTML report...
Generating XML report...

C:\Users\USER\Documents\KwangWoon\ROBIT\C++
되었습니다.
이 창을 닫으려면 아무 키나 누르세요...|
```

ocp.cpp 동작 결과

3. Liskov Substitution Principle - 자식 클래스는 부모 클래스를 대체할 수 있어야 한다
LSP의 내용은 정리하면 다음과 같다.

자식 클래스는 단순히 부모 클래스를 문법적으로 상속받는 것에서 그쳐서는 안 되며, 프로그램 안에서 부모 클래스가 사용되던 자리에 자식 클래스를 그대로 가져다 놓아도 프로그램이 원래 의도한 대로 정상적으로 동작해야 한다. 즉, 자식 클래스는 부모 클래스가 가지는 동작 방식까지 함께 지켜야 한다.

예를 들면, person 클래스에 walk()라는 기능이 있고, 이를 상속받은 student라는 자식 클래스가 있다고 하자. 만약 student가 walk()를 오버라이딩하면서 이유 없이 예외를 던지거나 아무 동작도 하지 않도록 구현한다면, person 대신 student를 사용하는 순간 프로그램이 깨지게 된다. 이는 student가 person의 역할을 온전히 대체하지 못한다는 뜻이며, LSP를 위반한 것이다.

이러한 원칙의 이유는 다음과 같다.

LSP를 지키지 않으면, 겉보기에는 상속 관계로 잘 짜인 코드처럼 보여도, 실제로는 자식 클래스마다 개별적으로 타입을 확인하고 예외 처리를 해줘야 하는 상황이 발생한다. 이렇게 되면 다형성을 이용해 코드를 단순화하려던 상속의 의미가 사라지고, OCP에서 말한 "확장에 열려 있는" 구조도 함께 무너지게 된다. 결국 LSP는 상속과 다형성이 실제로 안전하게 동작하기 위한 전제 조건으로 작용하는 것이기 때문이다.

```
Bird is eating
Sparrow is flying
Bird is eating

C:\Users\USER\Documents\KwangWoon\ROBIT
되었습니다 .
이 창을 닫으려면 아무 키나 누르세요 ...|
```

lsp1.cpp 동작 결과

```
Area: 20
Area: 25

C:\Users\USER\Documents\Kwang
되었습니다 .
이 창을 닫으려면 아무 키나 누
```

lsp2.cpp 동작 결과

4. Interface Segregation Principle - 클래스는 사용하지 않을 기능을 강요받지 말아야 한다

ISP의 내용은 정리하면 다음과 같다.

하나의 인터페이스에 너무 많은 기능을 모아두면, 그 인터페이스를 구현하는 클래스는 실제로 필요하지 않은 기능까지 억지로 구현해야 하는 상황이 생긴다. 따라서 하나의 크고 포괄적인 인터페이스보다는, 사용 목적에 맞게 잘게 나뉜 여러 개의 인터페이스로 설계하는 것이 바람직하다.

예를 들면, activity라는 하나의 인터페이스 안에 study(), exercise(), reading() 기능이 모두 들어있다고 하자. 만약 person 중 일부는 공부만 하고 운동이나 독서는 하지 않는 경우라면, 이 person은 필요 없는 exercise(), reading() 기능까지 억지로 구현하거나 빈 채로 남겨두어야 한다. 이럴 때는 activity를 studyable, exercisable, readable처럼 역할별로 잘게 나누어, 각 person이 자신에게 필요한 인터페이스만 선택적으로 구현하도록 해야 한다.

이러한 원칙의 이유는 다음과 같다.

불필요한 기능까지 강제로 구현하게 되면 코드가 불필요하게 복잡해지고, 사용하지도 않는 기능 때문에 인터페이스가 변경될 때마다 관련 없는 클래스들까지 영향을 받게 된다. 반대로 인터페이스를 잘게 나누어두면, 각 클래스는 자신과 관련 있는 인터페이스의 변경에만 영향을

반게 되므로 결합도가 낮아지고 코드의 유연성이 높아진다.

```
Employee is working
Employee is eating
Robot is working

C:\Users\USER\Documents\KwangWoon\ROBIT\
되었습니다.
이 창을 닫으려면 아무 키나 누르세요 ...|
```

isp.cpp 동작 결과

5. Dependency Inversion Principle - 고수준 모듈이 저수준 모듈에 의존해서는 안 된다
DIP의 내용은 정리하면 다음과 같다.

상위 수준에서 전체적인 흐름을 담당하는 모듈이, 세부적인 구현을 담당하는 하위 수준의 모듈에 직접 의존해서는 안 된다. 고수준 모듈과 저수준 모듈 모두 그 사이에 존재하는 추상화(인터페이스)에 의존하도록 설계해야 하며, 이렇게 되면 실제 의존의 방향이 하위 모듈에서 상위 모듈 쪽의 추상화로 "역전"된다.

예를 들면, person 클래스가 공부라는 역할을 수행할 때, 특정한 study 클래스(예: mathStudy)에 직접 의존하게 되면, 나중에 공부 방식이 englishStudy로 바뀔 때마다 person 클래스 내부 코드를 계속 수정해야 한다. 이 대신 person은 studyable이라는 추상화된 인터페이스에만 의존하도록 하고, mathStudy와 englishStudy가 각각 이 인터페이스를 구현하도록 하면, person 코드는 전혀 수정하지 않고도 어떤 구체적인 study 구현체든 갈아 끼울 수 있게 된다.

이러한 원칙의 이유는 다음과 같다.

고수준 모듈이 저수준의 구체적인 구현에 직접 의존하면, 저수준 모듈이 바뀔 때마다 상위의 핵심 로직까지 함께 수정해야 하므로 변경에 취약해진다. 반면 추상화를 매개로 의존하게 되면, 구체적인 구현체는 언제든지 자유롭게 교체할 수 있고, 테스트를 할 때도 실제 구현 대신 가짜 구현(mock)을 넣어 손쉽게 검증할 수 있다.

```
Fan is spinning
Fan is stopping

C:\Users\USER\Documents\KwangWoon\ROBI
되었습니다.
이 창을 닫으려면 아무 키나 누르세요 ...|
```

div.cpp 동작 결과

6. 결론

SOLID 원칙은

SOLID 원칙은 결국 유연하게 변경할 수 있고, 활용도가 높은 코드를 만들기 위한 다섯 가지 지침으로, 이를 따라 코드를 작성하는 것이 안전하다. 그리고, 절대적인 규칙이라기보다, 상황에 맞게 선택적으로 적용하여 따르는 것이 적절하다.